



Python Course 2025/26/2

Lecture 9: AI, ML, Data Science, and Modern DataFrames



Course coordinator: Dr Mate Tejfel

Lecturer: Zoltan Kegli



Lecture Overview

Agenda

- Separate AI, ML, deep learning, generative AI, and data science
- See how data projects move from questions to deployed results
- Map the modern Python stack: JupyterLab, pandas, Polars, plotting, ML
- Use current pandas 3.0 patterns for ingestion, null handling, and safe mutation
- Understand when Polars is worth the mental-model shift

Learning objective

Move from buzzwords and notebooks-by-habit to a clearer mental model of modern data work: what the terms mean, which tools fit which job, and how to write safer dataframe code.

By the end, you can

Explain the AI/ML/data science landscape, choose between pandas and Polars for common scenarios, and recognize the main pandas 3.0 habits you should actually trust in 2026.

Concepts

Workflow

JupyterLab

pandas 3.0

Polars

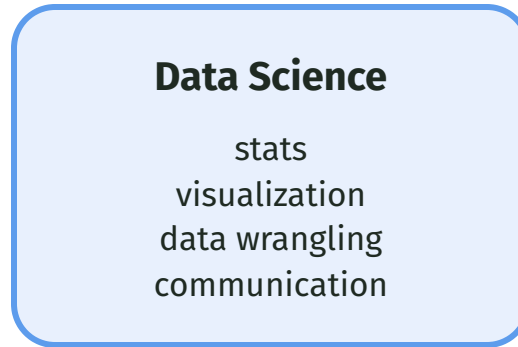
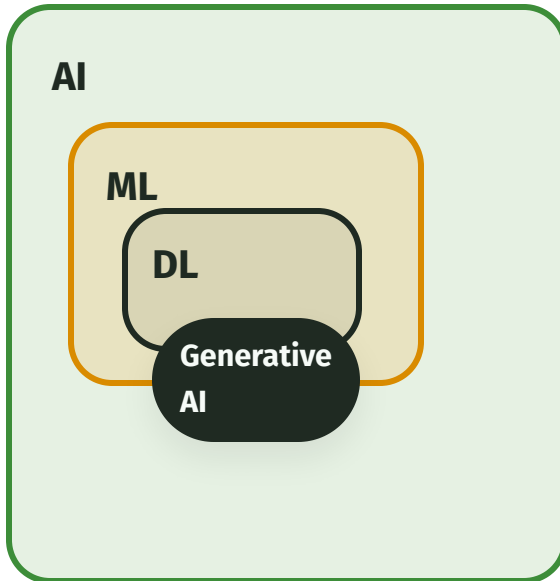
Lecture 9 is the bridge from "AI words" to hands-on dataframe work.



AI, ML, and Data Science

A mental map before we touch the tools

The Taxonomy: Related, But Not Identical



Data science overlaps with AI/ML when it uses predictive or generative models.

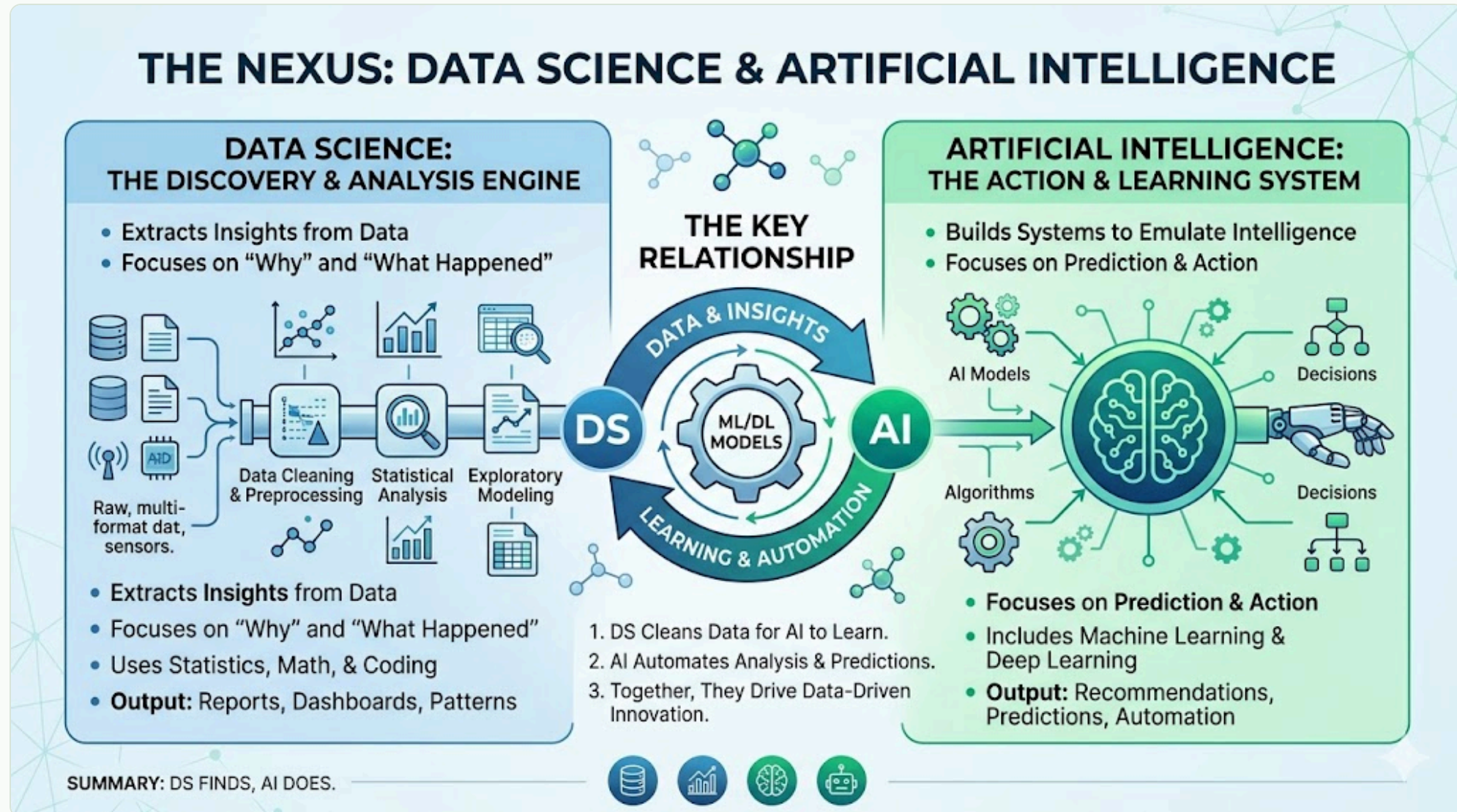
Use the terms precisely

- **AI:** the broad goal of building systems that act intelligently
- **ML:** AI methods that learn patterns from data
- **DL:** ML with layered neural networks
- **Generative AI:** systems that produce new content; flagship tools are usually deep nets, but classical or hybrid generators (templates, grammars, statistical models) still count
- **Data Science:** framing questions, preparing data, analyzing results, and communicating decisions, often using ML as one tool

Fun history fact

The term `Artificial Intelligence` was coined for the Dartmouth workshop in 1956, not in the deep-learning era people usually associate with AI.

Data Science and AI (reference diagram)





AI Capabilities by Problem Type

Perception

Vision, sensors, detection, segmentation, tracking

Language

Classification, translation, summarization, question answering

Reasoning

Planning, search, ranking, decision support, constraint solving

Retrieval

Search, memory systems, recommendations, knowledge grounding

Generation

Text, image, audio, video, code, synthetic data

Action

Robotics, control loops, autonomous behavior, physical systems

One product, many capabilities

A modern assistant can combine retrieval, language generation, reasoning, and user feedback in the same workflow.

That is why "AI product" is broader than "one model."

Before choosing a model or library, ask: **what capability do we actually need?**



Three Core Learning Paradigms

Supervised

Learn from labeled examples.

Example: predict spam vs not spam from emails with known labels.

Unsupervised

Find structure without target labels.

Example: cluster customers by behavior or detect anomalies.

Reinforcement

Learn through actions, rewards, and repeated feedback.

Example: game playing, robotics, sequential decision tasks.

Hype vs. Reality: Generative AI gets the spotlight, but classical ML still powers many core business decisions.

The Advantage: For structured-data problems, simpler models are usually faster, cheaper, and easier to explain and govern.

The Golden Rule: Start with the simplest model that solves the problem, and only scale up when the data or use case truly demands it.



Why This Matters Outside the Classroom

Transport

Dynamic pricing, maintenance prediction, routing, and demand forecasting.

Healthcare

Medical imaging support, triage analytics, and resource planning.

Retail & Finance

Fraud detection, recommendations, customer segmentation, pricing, and risk analysis.

Important reality check

Useful systems need far more than a model: data collection, labeling, evaluation, interfaces, monitoring, and often human review.



CRISP-DM: Still the Skeleton of Many Projects

1. Business

What decision or problem are we trying to improve?

2. Data

What data exists, and how trustworthy is it?

3. Prepare

Clean, join, reshape, validate, and document the dataset.

4. Model

Train or configure the analytical method that fits the task.

5. Evaluate

Check quality against both metrics and business meaning.

6. Deploy

Ship results into reports, apps, services, or monitoring loops.

What 2026 teams add

- Version control for data and code
- Tests, review, and reproducible environments
- Monitoring for quality drift after deployment
- MLOps or analytics ops on top of the core loop

Fun workflow fact

CRISP-DM comes from the late 1990s, but the loop still survives because most real projects still bounce between goals, messy data, evaluation, and deployment.



Checkpoint: AI, ML, and Data Science

Q1
A team groups customers into segments without any target labels. Which learning paradigm fits best?

- A) Supervised learning
- B) Unsupervised learning
- C) Reinforcement learning
- D) Mystical spreadsheet telepathy

Show Answer

Q2
A hospital team cleans records, visualizes trends, and uses a model to forecast bed demand. Which label is most accurate?

- A) Only deep learning
- B) Only data engineering
- C) Data science, possibly including ML
- D) A printer driver with ambition

Show Answer



Modern Python Data Workflow

From workspace to dataframe to first answer



The Stack as a Workflow

Learning objective

See the modern Python data stack as a workflow from question to trustworthy answer: workspace, dataframe tools, explanation, and only then modeling if needed.

By the end of this section, you can

- Explain where JupyterLab, pandas, Polars, plotting, and ML libraries fit
- Describe the fast loop: load, inspect, answer, refactor
- Recognize the dataframe as the main working table
- Use a first filtered question to build confidence in a dataset

Workspace

JupyterLab, notebooks, scripts, terminals

Working table

pandas by default; Polars when lazy/selective pipelines matter

Explanation

summary tables, plots, and short written conclusions

Modeling

scikit-learn, PyTorch, and friends after the data is trustworthy

JupyterLab

pandas

Polars

plots

ML

In many classroom and business tasks, the work ends in a table, chart, or decision note. Modeling is a later step, not the starting point.



Notebook First, Module Next

The fast loop for data work

Load

Open a local CSV or parquet file and define what one row means.

Inspect

Check `head()`, dtypes, and missing values before transforming.

Answer

Filter, group, summarize, and plot to answer one concrete question.

Extract

Move stable logic into modules, tests, and reusable scripts.

Good notebook habits

- Keep one question or step per cell
- Restart and rerun from the top when the notebook stabilizes
- Write short markdown notes about assumptions and findings

What should leave the notebook

- Reusable cleaning functions
- Shared pipeline logic and package code
- Tests, packaging, and deployment details



The DataFrame Is the Working Table

How to think about it

- **Series:** one labeled dimension, often one column or one summary result
- **DataFrame:** multiple aligned series in one table
- **Rows:** records such as missions, observations, or events
- **Columns:** variables such as agency, destination, or cost
- **Dtypes:** part of the meaning, not just an implementation detail

In the ``space_missions`` example, one row is one mission. That mental model makes filtering and grouping much easier to explain.

A tiny dataframe example

Species	Island	Records	Sampled
Adelie	Torgersen	152	True
Chinstrap	Dream	68	True
Gentoo	Biscoe	124	True

A `Series` might be the `records` column alone, or the result of a summary like `value_counts()`.



First Real Table: Read, Inspect, Ask

```
import pandas as pd

missions = pd.read_csv(
    "space_missions.csv",
    dtype_backend="pyarrow",
)

moon_missions = missions.loc[
    missions["destination"] == "Moon",
    ["mission_name", "launch_year", "agency", "cost_million_usd"],
].sort_values("launch_year")

agency_counts = missions["agency"].value_counts()
```

Read a real table, inspect it, and ask one focused question before doing heavier transformations.

Patterns to recognize quickly

- `head()`, `dtypes`, and null checks after loading
- `df.iloc[0]` for a first positional row
- `df.loc[mask, cols]` for clear filtering
- `value_counts()` for a first categorical summary

Question-first analysis

"Which Moon missions are in this dataset?" and "Which agency appears most often?" are good first questions because they build confidence in both the table and the access patterns.



Checkpoint: Modern Python Data Workflow

Q3

After loading a new CSV into a dataframe, what is the best next move?

- A) Inspect the first rows, dtypes, and missing values
- B) Start model training immediately
- C) Ignore the table and only rename files
- D) Choose chart colors before reading the data

Show Answer

Q4

In the `space_missions` dataframe, one row most naturally represents what?

- A) One mission record
- B) One Python package
- C) One notebook tab
- D) One plotting theme

Show Answer



pandas in Practice

From raw tables to filtered, cleaned, and grouped answers

What pandas Is Good At

Learning objective

Use pandas as the standard tool for tabular data work: load a dataset, inspect it, transform it, and answer concrete questions with readable code.

By the end of this section, you can

- Read a CSV into a dataframe and inspect its structure
- Filter rows and columns to answer a focused question
- Create new columns and lookup joins from clear, auditable rules
- Clean text-heavy or partially missing columns explicitly
- Summarize grouped or time-based patterns into decision-ready tables

Typical problems pandas solves

- **Exploration:** inspect columns, dtypes, and missing values
- **Selection:** keep only the rows and columns relevant to the question
- **Feature engineering:** derive labels, bands, dates, or rates from raw columns
- **Integration:** attach metadata from another table with a validated join
- **Cleaning:** turn messy strings into useful typed columns
- **Reporting:** build grouped summaries that someone can actually discuss

CSV filtering cleaning groupby joins

Pandas shines when the data is tabular, labeled, and small-to-medium enough to fit in memory while you iterate.



Scenario 1: Inspect the Table Before Touching It

```
import pandas as pd

missions = pd.read_csv(
    "space_missions.csv",
    dtype_backend="pyarrow",
)

missions.head(5)
missions.dtypes
missions.isna().sum()
```

The first task is not clever analysis. It is understanding the table: what one row means, which columns are numeric or text, and where missing values may matter later.

What to inspect immediately

- What does one row represent?
- Which columns are text, numeric, or boolean?
- How many missing values does each column have?
- Which columns look like identifiers, categories, or measurements?

Good analysis starts with structure, not with a clever one-liner. In these demos, `dtype_backend="pyarrow"` keeps nullable columns easy to reason about.



Scenario 2: Filter to the Relevant Rows

```
moon_missions = missions.loc[
    missions["destination"] == "Moon",
    [
        "mission_name",
        "launch_year",
        "agency",
        "cost_million_usd",
    ],
].sort_values("launch_year")
```

Pick Moon rows, keep four useful columns, then sort by year for a clean answer table.

Pattern: `.loc[rows, columns]` keeps both filters explicit.

Moon missions in the current dataset

Mission	Year	Agency	\$M
Apollo 8	1968	NASA	2500
Apollo 11	1969	NASA	25400
Apollo 13	1970	NASA	25400
Chang'e 4	2018	CNSA	170
Chandrayaan-2	2019	ISRO	140
Chang'e 5	2020	CNSA	200
Artemis I	2022	NASA	4100
Chandrayaan-3	2023	ISRO	75
Artemis II	2026	NASA	4500

All 9 Moon rows in `space_missions.csv`, sorted by `launch_year`.



Scenario 3: Derive Meaningful Columns Safely

```
missions["mission_style"] = "uncrewed"
missions.loc[missions["crew_size"] > 0, "mission_style"] = "crewed"

missions["distance_band"] = "local"
missions.loc[missions["distance_km"] ≥ 384_400, "distance_band"] = "lunar_or_beyond"
missions.loc[missions["distance_km"] ≥ 1_000_000, "distance_band"] = "deep_space"
```

A typical dataframe task is to turn raw numeric columns into labels that are easier to group, plot, and explain.

What this gives us

- Apollo 11 -> crewed, lunar_or_beyond
- Voyager 1 -> uncrewed, deep_space
- ISS Expedition 1 -> crewed, local
- James Webb -> uncrewed, deep_space

This is feature engineering in dataframe form. Use explicit column assignment and `.loc` updates when the rule depends on a condition.



Scenario 4: Parse a Messy Text Column

```
wkt_pattern = (  
    r"POINT Z \((?P<longitude>.*?) "  
    r"(?P<latitude>.*?) "  
    r"(?P<depth>.*?)\)"  
)  
  
coords = (  
    quakes["geometry"]  
    .str.extract(wkt_pattern)  
    .astype("float32[pyarrow]")  
)
```

The key idea is not the regex itself. The key idea is that the same parsing rule is applied to the whole column, producing new numeric columns in one step.

From one cell to three columns

Raw geometry cell	Extracted values
POINT Z (-169.6766 52.3338 10)	longitude=-169.6766 latitude=52.3338 depth=10

Prefer vectorized string methods over Python row loops when the same cleaning rule applies to every row.



Scenario 5: Build a Summary Table

```
destination_summary = (
    missions[missions["destination"].isin(["Moon", "Mars", "ISS"])]
    .assign(cost_per_day=lambda df: (df["cost_million_usd"] / df["duration_days"]).round(2))
    .groupby("destination")
    .agg(
        mission_count=("mission_name", "size"),
        avg_cost_per_day=("cost_per_day", "mean"),
        crewed_missions=("mission_style", lambda s: (s == "crewed").sum()),
    )
    .sort_values("avg_cost_per_day", ascending=False)
)
```

This pipeline filters, derives, groups, and sorts to answer a reporting question.

What this summary answers

Which destinations are most expensive per day, and how many missions are crewed?

- **Moon:** count 9, avg/day 937.90, crewed 4
- **ISS:** count 1, avg/day 23.44, crewed 1
- **Mars:** count 9, avg/day 0.83, crewed 0

Why `groupby().agg()` matters:

it compresses many detailed rows into an answer table you can explain in one sentence.



Scenario 6: Attach Lookup Data with `merge()`

```
agency_info = pd.DataFrame(  
    {  
        "agency": ["NASA", "CNSA", "ISRO"],  
        "region": ["USA", "China", "India"],  
        "crew_program": ["yes", "planned", "planned"],  
    }  
)  
  
moon_with_agency = moon_missions.merge(  
    agency_info,  
    on="agency",  
    how="left",  
    validate="many_to_one",  
)
```

Real analysis often starts by attaching metadata from a small lookup table. Use `validate=` to catch broken join assumptions early.

Question

How do we add agency metadata without handwritten `if/else` rules?

Mission	Agency	Region	Crew program
Apollo 11	NASA	USA	yes
Chang'e 4	CNSA	China	planned
Chandrayaan-2	ISRO	India	planned
Artemis I	NASA	USA	yes
Artemis II	NASA	USA	yes

Best practice: if the right-hand table should have one row per key, say so with `validate="many_to_one"`.



Scenario 7: Handle Missing Values Explicitly

```
quake_alerts = (  
    quakes.loc[:, ["place", "mag", "mmi", "alert"]]  
    .assign(alert=lambda df: df["alert"].fillna("unknown"))  
    .dropna(subset=["mmi"])  
    .sort_values(["mmi", "mag"], ascending=[False, False])  
    .head(5)  
)
```

Not every missing value deserves the same fix. Fill a category only when an honest fallback exists; drop rows when the missing measurement would break the comparison.

Column-specific rule

Column	Missing	Action
alert	5 rows	fillna("unknown")
mmi	1 row	dropna(subset=["mmi"])

Best practice: do not blindly fill numeric missing values with `0`. Pick a domain-safe rule per column.



Scenario 8: Parse Time Columns Before Grouping

```
quakes["time_utc"] = pd.to_datetime(
    quakes["time"], unit="ms", utc=True
)

quakes_by_year = (
    quakes.assign(year=quakes["time_utc"].dt.year)
    .groupby("year")
    .agg(event_count=("id", "size"), max_mag=("mag", "max"))
    .sort_index()
    .tail(6)
)
```

Parse timestamps once, then use the `.dt` accessor for year/month/day logic. Avoid slicing raw integers or timestamp strings by hand.

What this summary answers

How many earthquakes are in the file each year, and what was the largest magnitude?

Year	Events	Max mag
2020	121	7.8
2021	157	8.2
2022	127	7.6
2023	147	7.8
2024	99	7.5
2025	100	8.8

Best practice: use `utc=True` for machine timestamps so time zones stay explicit from the start.



Checkpoint: pandas workflows

Q5

You want only Moon missions, selected columns, ordered by year. Best pattern?

A) `missions.loc[mask, cols].sort_values("launch_year")`

B) Turn the dataframe into a list first

C) Loop through rows manually

D) Randomly reorder the dataframe

Show Answer

Q6

Need mission counts and average duration per agency. Best operation?

A) `groupby("agency").agg( ... )`

B) `iloc[0]`

C) `str.extract( ... )`

D) `rename( ... )`

Show Answer



When Polars Is Worth It

Keep pandas for routine notebook work; switch when optimized pipelines matter



When pandas Is Enough, When Polars Is Worth It

Learning objective

Choose between pandas and Polars based on workload: use pandas for routine notebook work, and reach for Polars when query planning and execution strategy start to matter.

By the end of this section, you can

- Explain why pandas remains the default first choice
- Spot workloads where lazy execution makes Polars attractive
- Read a Polars query that filters, groups, and ranks
- Move results back into pandas when needed

pandas is enough when

- The data fits comfortably in memory
- You want quick notebook exploration
- The next step is plotting or scikit-learn
- The team already reads pandas fluently

Polars is worth it when

- You read large or wide CSVs but use only part of them
- You run heavy filter/select/group pipelines on one machine
- You want lazy planning before execution
- You need streaming or window-heavy transforms



Scenario 1: Large CSV, Small Answer Table

```
quake_query = (  
    pl.scan_csv("earthquakes.csv")  
    .select("net", "mag", "tsunami")  
    .filter(pl.col("mag").cast(pl.Float64) ≥ 6.0)  
    .group_by("net")  
    .agg(  
        pl.len().alias("quake_count"),  
        pl.col("mag").cast(pl.Float64).mean().alias("avg_magnitude"),  
        pl.col("tsunami").cast(pl.Int64).sum().alias("tsunami_events"),  
    )  
    .sort("quake_count", descending=True)  
)
```

This is the kind of workload where Polars starts to pay off: a wide CSV, only a few useful columns, and a grouped summary at the end.

What the summary tells us

- **us:** 1324 quakes, avg mag 6.38, tsunami 415
- **ak:** 8 quakes, avg mag 6.75, tsunami 8
- **nc:** 4 quakes, avg mag 6.40, tsunami 3

On a small classroom file, pandas would also work. Polars becomes worth trying when the same filter/select/group pattern is aimed at larger files and you want `scan_csv( ... )` to stay lazy until `collect()`.



Scenario 2: Rank Within Each Agency

```
missions = pl.read_csv("space_missions.csv")

ranked_missions = (
    missions.select("agency", "mission_name", "cost_million_usd")
    .with_columns(
        pl.col("cost_million_usd")
        .rank("dense", descending=True)
        .over("agency")
        .alias("cost_rank_within_agency")
    )
    .filter(pl.col("cost_rank_within_agency") ≤ 2)
    .sort(["agency", "cost_rank_within_agency"])
)
```

This example from the local mission dataset asks for a top-2 ranking within each agency, which is a natural place to introduce Polars expressions and window logic.

What this answers

- **NASA:** Apollo 11 rank 1, Artemis I rank 2
- **NASA/ESA:** James Webb rank 1, Hubble rank 2
- Smaller agencies keep only their most expensive mission

`over("agency")` is a window expression: rank inside each group without manually splitting the dataframe. If you already know pandas and only need one quick notebook answer, pandas is still fine. Polars starts to feel worth it when many expression-based per-group transforms stack up.



Scenario 3: Optimize in Polars, Deliver in pandas

```
ranked_missions_pd = ranked_missions.to_pandas()
# hand off to plotting / scikit-learn / older code

round_tripped = pl.from_pandas(ranked_missions_pd)
```

You do not have to choose one library forever. A practical workflow can do the heavy dataframe step in Polars and still hand the result to the pandas ecosystem.

Scenario	Better first pick
Notebook exploration, plotting, scikit-learn handoff	pandas
Large CSV with select/filter/group pipeline	Polars
Expression-heavy ranking or window logic	Polars
Heavy transform first, pandas ecosystem next	Polars -> pandas

Start with pandas by default. Add Polars when the workload justifies lazy planning, selective CSV scans, or heavier expression pipelines.



Checkpoint: pandas vs Polars

Q7

Your data fits in memory, you are exploring in a notebook, and the next step is plotting plus scikit-learn. Best first choice?

- A) pandas
- B) Polars, no discussion needed
- C) Rewrite everything in assembly
- D) A blockchain-powered CSV

Show Answer

Q8

You have a wide CSV, but the answer needs only a few columns after filtering before a grouped summary. Which Polars pattern is worth trying?

- A) `pl.scan_csv( ... ).select( ... ).filter( ... ).group_by( ... ).agg( ... ).collect()`
- B) Read everything eagerly and loop row by row in Python
- C) Convert the file to screenshots first
- D) Use `iloc[0]` and hope the first row explains the whole dataset

Show Answer



Lecture Summary

Workflow

- Start in a notebook or script with one concrete question
- Load, inspect, answer, then extract stable logic
- Modeling comes after the data is trustworthy

Working Table

- Treat the dataframe as the main working table
- Read, inspect, filter, derive, parse, summarize
- Use `.loc`, vectorized code, and clear pipelines

Tool Choice

- JupyterLab for fast exploration and teaching
- pandas by default for routine notebook workflows
- Polars when lazy scans or heavier pipelines justify it

The real skill is not memorizing library names. It is following a clean workflow from question to table to answer, then choosing the lightest tool that fits the workload.



Thank You for Your Attention

“Without data you're just another person with an opinion.” — W.
Edwards Deming

AI

Data Science

pandas

Polars